Multi-Modal Molecular Stacking Pipeline

OBJECTIVE: **To develop and train a Graph Neural Network (GNN) for highly accurate molecular 5 binary properties classification**

TEAM: **Data AImers**

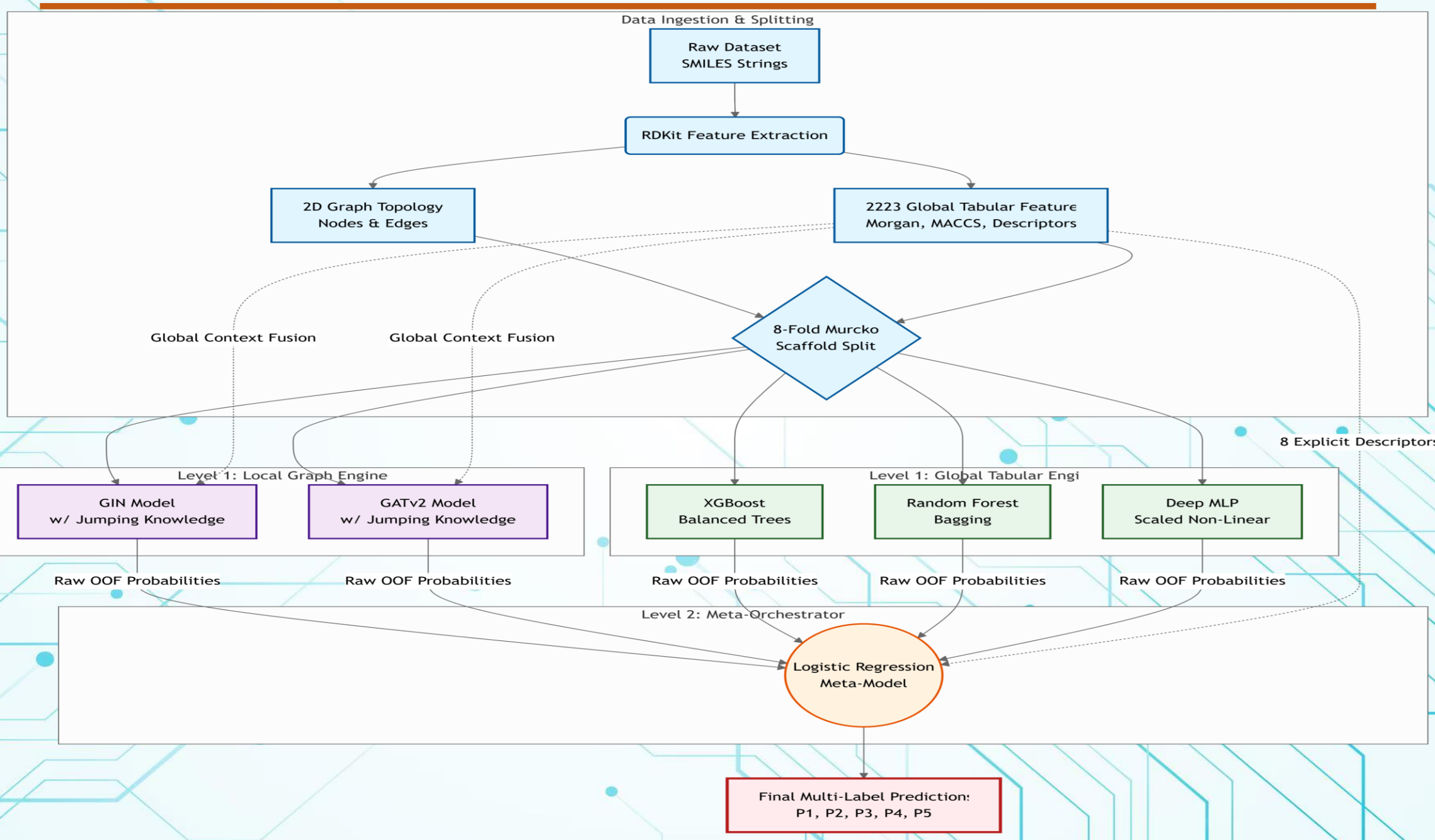
DESCRIPTION: **Standard Graph Neural Networks (GNNs) often fail to capture global molecular properties (like total weight or solubility), while traditional Tree models struggle to understand 2D structural topology.**

We built a multi-level stacking pipeline that separates local bond extraction from global property analysis, merging them via a rank-based meta-model.

CORE CAPABILITIES:

1. **Multi-View Feature Extraction: Simultaneously extracts 2D graph topology (nodes/edges) and 2223 global tabular features (Morgan Fingerprints, MACCS Keys, RDKit Descriptors).**
2. **Jumping Knowledge Integration: Prevents deep GNN over-smoothing by concatenating local atom representations across all message-passing layers.**

High-level System Architecture



Technical Stack

❖ *FEATURE EXTRACTION*

***TECHNOLOGY USED*:** RDKit & rdFingerprintGenerator

- ✓ **Use:** Extracts 2D graph topology and 2223 global tabular features (Morgan, MACCS, Descriptors).
- ✓ **Edge:** Dummy Atom Fallback ('C') silently handles broken SMILES strings, guaranteeing zero pipeline crashes.

❖ *LOCAL GRAPH ENGINES*

***TECHNOLOGY USED*:** PyTorch Geometric (GIN & GATv2)

- ✓ **Use:** Message-passing networks that learn local bond structures and reactive zones.
- ✓ **Edge:** "Jumping Knowledge" integration completely prevents over-smoothing on complex molecular structures.

❖ *GLOBAL TABULAR ENGINES*

***TECHNOLOGY USED*:** XGBoost, Random Forest, Deep MLP

- ✓ **Use:** Processes the dense, 2223-dimensional global chemical fingerprints.
- ✓ **Edge:** Combines three distinct paradigms (Boosting, Bagging, Deep Non-Linear) to capture macro-interactions graphs cannot see.

❖ *META-ORCHESTRATOR*

***TECHNOLOGY USED*:** SciPy (rankdata) & Logistic Regression

- ✓ **Use:** Synthesizes all 5 Level-1 models into a final multi-label consensus.
- ✓ **Edge:** Percentile Rank Transformation bypasses probability calibration clashes, completely immunizing the ensemble against leaderboard distribution shifts.

❖ *LOSS & OPTIMIZATION*

***TECHNOLOGY USED*:** Focal Loss & Cosine AnnealingLR

- ✓ **Use:** Manages gradient updates and penalizes prediction errors.
- ✓ **Edge:** Dynamic class weights and Focal Loss (`gamma=2.0`) force the network to prioritize rare active molecules, maximizing the MCC metric.

❖ *VALIDATION STRATEGY*

***TECHNOLOGY USED*:** Murcko Scaffold Split (8-Fold)

- ✓ **Use:** Splits the dataset based on core 2D ring structures rather than random sampling.
- ✓ **Edge:** Forces models to generalize to unseen chemical backbones, perfectly simulating the hidden Private Leaderboard.

Edge Case Resolution

PROBLEM	SYSTEM COMPONENT	OUTCOME
Invalid/Broken SMILES Strings	Strings Dummy Atom Fallback (Chem.MolFromSmiles('C'))	Silently replaces broken dataset molecules with a single Carbon atom, ensuring array dimensions remain stable and preventing PyTorch DataLoader crashes.
Extreme Target Imbalance	Focal Loss (gamma=2.0) + Dynamic POS Weights	Dynamically scales the penalty for missing rare active molecules, forcing the neural networks to focus on the minority class to maximize the MCC metric.
BatchNorm Crashing on Small Batches	(drop_last=True) in DataLoader	Prevents fatal PyTorch errors when the final training batch contains only 1 molecule, which mathematically breaks standard deviation calculations in 1D Batch Normalization.
MCC Optimizer False Negatives	Clamped Search Space (0.45 to 0.75)	Prevents the dynamic threshold optimizer from aggressively chasing perfect precision (e.g., threshold > 0.85) which would tank the MCC score through excessive False Negatives.